```
            q.dignity = dignity;
            q.ego = ego;
            if (copy_to_user((query_arg_t *)arg, &q,
sizeof(query_arg_t)))
            {
                return -EACCES;
            }
            break;
        case QUERY_CLR_VARIABLES:
            status = 0;
            dignity = 0;
            ego = 0;
            break;
        default:
            return -EINVAL;
    }

    return 0;
}
```

And finally, the corresponding invocation functions from the application *query_app.c* would be as follows:

```
#include <stdio.h>
#include <sys/ioctl.h>

#include "query_ioctl.h"

void get_vars(int fd)
{
    query_arg_t q;
    if (ioctl(fd, QUERY_GET_VARIABLES, &q) == -1)
    {
        perror("query_apps ioctl get");
    }
    else
    {
        printf("Status : %d\n", q.status);
        printf("Dignity: %d\n", q.dignity);
        printf("Ego    : %d\n", q.ego);
    }
}
void clr_vars(int fd)
{
    if (ioctl(fd, QUERY_CLR_VARIABLES) == -1)
    {
        perror("query_apps ioctl clr");
    }
}
```

The complete code of the above three files is available on the magazine's website at *http://linuxforu.com/article_source_code/aug11/* ldd_code.*tar.bz2*, where the required *Makefile* is also present. Try it out with the following operations:

- Build the '*query_ioctl*' driver (*query_ioctl.ko* file) and the application (*query_app* file) by running make, using the provided Makefile.
- Load the driver using *insmod query_ioctl.ko*.
- With appropriate privileges and command-line arguments, run the application *query_app:*
  - *./query_app #* To display the driver variables
  - *./query_app -c #* To clear the driver variables
  - *./query_app -g #* To display the driver variables
  - *./query_app -s #* To set the driver variables (not mentioned above)
- Unload the driver using rmmod *query_ioctl*.

Defining the *ioctl()* commands

"Visiting time is over," yelled the security guard. Shweta thanked her friends since she could understand most of the code now, including the need for *copy_to_user()*, as learnt earlier. But she wondered about *_IOR, _IO*, etc, which were used in defining commands in *query_ioctl.h*. These are usual numbers only, as mentioned earlier for an *ioctl()* command. Just that, now additionally, some useful command related information is also encoded as part of these numbers using various macros, as per the Portable Operating System Interface (POSIX) standard for *ioctl*. The standard talks about the 32-bit command numbers, formed of four components embedded into the [31:0] bits:

1. The direction of command operation [bits 31:30]—read, write, both, or none—filled by the corresponding macro (*_IOR, _IOW, _IOWR, _IO*).
2. The size of the command argument [bits 29:16]—computed using *sizeof()* with the command argument's type—the third argument to these macros.
3. The 8-bit magic number [bits 15:8]—to render the commands unique enough—typically an ASCII character (the first argument to these macros).
4. The original command number [bits 7:0]—the actual command number (1, 2, 3, …), defined as per our requirement—the second argument to these macros.

Check out the header *<asm-generic/ioctl.h>* for implementation details. END

By: Anil Kumar Pugalia

The author is a freelance trainer in Linux internals, Linux device drivers, embedded Linux and related topics. Prior to this, he was at Intel and Nvidia. He has been working with Linux since 1994. A gold medallist from the Indian Institute of Science, Linux and knowledge sharing are two of his many passions. Creating and playing with open source hardware is one of his hobbies. Learn more about him at http://*profession.sarika-pugs.com*, including information about his open source hardware freak-outs, and his various Linux related training sessions and workshops. He can be reached at *email@sarika-pugs.com*.